

LISTA DE EXERCÍCIOS 2.5

1. Assuma que um sistema operacional mapeia threads de nível de usuário para o kernel usando o modelo muitos-para-muitos e que o mapeamento seja feito por meio de LWPs. Além disso, o sistema permite que desenvolvedores criem threads de tempo real para uso em sistemas de tempo real. É necessário atrelar (*bind*) uma thread de tempo real a um LWP? Explique.

Resposta: Sim. Tempo é crucial em aplicações de tempo real. Se um thread é marcado como de tempo real, mas não é atrelado a um LWP, a thread pode ter que esperar para ser atrelada a um LWP antes de ser executada. Considere que uma thread de tempo real está em execução (é atrelada a um LWP) e então é bloqueada (i.e. deve realizar I/O, foi interrompida por uma thread de tempo real de maior prioridade, está esperando por um lock de exclusão mútua, etc). Enquanto a thread de tempo real está bloqueada, o LWP para o qual foi atrelada foi atribuído a outra thread. Quando a thread de tempo real foi escalonada para executar novamente, ela deve primeiro esperar para ser atrelada a um LWP. Pela ligação de um LWP a uma thread de tempo real, está se assegurando que a thread estará apta a executar com retardo (delay) mínimo uma vez que é escalonada.

2. Explique o princípio de cancelamento de *threads*.

Resposta: em um SO, o cancelamento de *threads* permite a sua interrupção antes que ela tenha terminado em seu ciclo de execução normal. Existem duas abordagens:

- Cancelamento assíncrono: uma *thread* termina a *thread* alvo imediatamente
- Cancelamento por deferimento: permite a *thread* alvo que ela periodicamente verifique se deve ser cancelada

3. Discorra sobre tratamento de sinais em SO.

Resposta: um sinal pode ser gerado por uma interrupção de software gerada por um processo. Nesse contexto, o SO usa sinais para reportar situações excepcionais a um processo em execução. Nesse caso, temos sinalização síncrona, pois a sinalização é entregue ao mesmo processo que ocasionou a interrupção de software (por exemplo, devido a uma exceção gerada por acesso ilegal a memória ou divisão por zero). No caso assíncrono, o sinal é gerado por um evento externo ao processo em execução (ex. CTRL+C, expiração de timer, SIGKILL etc – vide manual – *man signal*). Caso o programador antecipe um evento que ocasiona um sinal, pode ser definido uma função *handler* a qual será executada pelo SO quando um tipo particular de sinal ocorre. Caso contrário, o sinal por padrão é tratado pelo kernel.

Leitura complementar:

https://scc.ustc.edu.cn/zhsc/sugon/intel/compiler_f/main_for/bldaps_for/lin/bldaps_sig handl.htm

<https://www.tutorialspoint.com/signals-and-signal-handling>

4. O que são *thread pools* e onde são comumente usados?

Resposta: *thread pools* são grupos de threads (um *pool*), os quais aguardam trabalho. Comumente relacionado ao modelo M:N (muitos para muitos) ou modelo em dois níveis, no

qual são criados *pools* de threads de kernel (*pools* de LWPs – ponto de entrada para espaço de usuário, o qual é atrelado a uma *thread* de kernel). Como as *threads* de kernel criadas no *pool*, após a execução de uma *thread* de usuário são devolvidas para o *pool* (ao contrário de serem destruídas), geralmente são um pouco mais rápidas para servir uma *requisição*, quando comparado a necessidade de se criar uma nova *thread*. Com isso, limita-se o número de *threads* na(s) aplicação(ões) ao tamanho do *pool*.

5. O que é *thread local storage* (TLS)?

Resposta: geralmente, *threads* pertencentes a um processo compartilham os dados do processo. Em algumas circunstâncias, se permite que cada *thread* tenha sua **própria cópia dos dados**, os quais com a evolução das atividades da *thread*, podem mudar. Tais dados são chamados de thread-local storage (ou TLS). O TLS não pode ser confundido com uma variável local. Variáveis locais são visíveis apenas durante a invocação de uma única função. Os dados de TLS são visíveis entre invocações de função. A diferença é que o TLS é único a cada *thread*. Esse recurso é suportado pela maioria das bibliotecas (ex. Pthreads, Windows, Java) e é útil quando não se tem controle sobre o processo de criação de *thread* (i.e. ao usar um *pool*)

6. O que são ativações de escalonador? Destaque o papel dos *upcall handlers* em sua resposta.

Resposta: são interrupções (chamadas nesse contexto de *upcalls*) encaminhadas pelo kernel com o intuito de informar à aplicação sobre certos eventos. Esta comunicação permite que uma aplicação mantenha o número correto de *threads* de kernel (ex. por exemplo, desalocando *threads* de espaço de usuário que por ventura sejam bloqueadas – como em E/S). Na biblioteca de *threads*, as *upcalls* são tratadas na biblioteca de *threads* por *upcall handlers* (um escalonador de processo em espaço de usuário – vide sl. 22 e 23).